

Cross-Generator Detection of AI-Generated Product Reviews: A Leave-One-Generator-Out Study with Contrastive and Adversarial Robustness Objectives

ECS 111 Final Project

June 2026

Abstract

A detector for AI-written reviews is only useful if it works on text from models it never trained on. Most prior work trains and tests on the same generator, which overstates real performance. We instead measure the *cross-generator gap*: the drop in detection F1 on a generator held out of training. We build a balanced corpus of about 20,000 Amazon reviews, pairing roughly 10,000 confidently human reviews (written before ChatGPT) with about 10,000 AI reviews from four generators (GPT-5 mini, DeepSeek-V4-Flash, Gemma-4-31B-it, Qwen3.6-35B-A3B). All AI reviews cover the same products with human-like lengths, so the test isolates writing style. Using a leave-one-generator-out protocol over four rotations, we train a DeBERTa-v3 detector with optional supervised-contrastive and gradient-reversal adversarial heads, and compare against TF-IDF and frozen-DeBERTa baselines. Fine-tuned DeBERTa generalizes well (cross-generator F1 0.9839 versus 0.9558 for TF-IDF). The contrastive objective is best overall: it raises cross-generator F1 to 0.9875 and halves the gap from 0.0095 to 0.0048. The adversarial objective gives the smallest gap (0.0015) but lowers accuracy.

1 Introduction

Large language models can write fluent product reviews, creating an incentive for review fraud and a need to detect AI-generated reviews. A detector is only useful if it works on models it was not trained on, since new generators appear constantly and an attacker can pick one the defender has not seen. Most prior work trains and tests on the same generator, which measures memorization of a fixed model fingerprint rather than generalization, and overstates real performance.

We measure the quantity that matters for deployment: the *cross-generator gap*, the drop in F1 when a detector is tested on a generator absent from training. We treat the task as human-versus-AI classification of Amazon reviews under a leave-one-generator-out (LOGO) protocol. With four generators we run four rotations; each holds out one generator as the cross-generator test set and trains on humans plus the other three.

Our contributions are: (1) a controlled corpus where length and topic are fixed across generators, so the gap reflects style; (2) four detector variants on a shared DeBERTa-v3 encoder (cross-entropy, contrastive, adversarial, and both); and (3) a full LOGO evaluation against two baselines, with a per-generator breakdown. Our main finding is that fine-tuned DeBERTa already generalizes well, and a supervised contrastive objective improves cross-generator F1 while halving the gap.

Table 1: Final corpus composition after cleaning.

Source	Count
Human reviews	9,969
GPT-5 mini	2,490
DeepSeek-V4-Flash	2,499
Gemma-4-31B-it	2,500
Qwen3.6-35B-A3B	2,498
Total	19,956

2 Related Work

Machine-generated text detection. Prior approaches include supervised classifiers, zero-shot likelihood scoring, and watermarking. Supervised detectors are accurate in-distribution but degrade when the test text comes from a different model, decoder, or domain. We isolate and measure one axis of that degradation: the change of generator.

Fake review detection. Recent work studies AI-generated reviews and benchmarks [5, 6, 7], but usually does not separate the training generator from the test generator. Our LOGO design never lets a generator appear in both, so the reported gap is a clean estimate of generalization to an unseen model.

Robust representations. Two ideas motivate our variants. Supervised contrastive learning [3] pulls same-label examples together, which should cluster all AI reviews regardless of source. Domain-adversarial training with a gradient reversal layer [4] trains the encoder so an auxiliary head cannot recover the source generator, removing generator-specific cues. Our backbone is DeBERTa-v3 [2].

2.1 Justification of novelty

AI fake-review detection is not new, but cross-generator robustness is understudied. Most papers train and test on the same generator, which inflates reported performance [5, 6, 7]. Our project is novel in three ways. First, we explicitly evaluate cross-generator robustness by training on some AI models and testing on completely different ones; no prior review-detection work structures its evaluation this way. Second, we combine two techniques, contrastive loss and a generator-adversarial head, to resist generator shift; neither has been applied to this problem before. Third, we run an ablation to identify which component drives robustness, giving an interpretable account of what makes the model work.

3 Methodology

3.1 Data sources

The corpus combines human reviews from the McAuley-Lab Amazon Reviews 2023 dataset [1], restricted to text written before ChatGPT’s release, with AI reviews we generated from four models. Table 1 gives the final counts.

Table 2: The four leave-one-generator-out rotations.

Rotation	Held out	Training generators
0	gpt5mini	deepseek + gemma + qwen
1	deepseek	gpt5mini + gemma + qwen
2	gemma	gpt5mini + deepseek + qwen
3	qwen	gpt5mini + deepseek + gemma

3.2 Human reviews

We stream the multi-gigabyte category files and sample uniformly with reservoir sampling. Each review must pass three filters: a pre-ChatGPT timestamp (before 2022-11-30), giving high confidence it is human; a length of 20 to 400 words; and an English-only check via `langdetect`. We draw 1,250 reviews from each of eight product categories. Human word counts have mean 60.6 and median 43, and ratings skew toward 5 stars, which we carry into the AI prompts.

3.3 AI reviews

We use four distinct generators: `gpt5mini` (OpenAI API), `deepseek`, `gemma`, and `qwen` (the last three via the HuggingFace Inference Providers router). We substituted DeepSeek-V4-Flash for the proposal’s IBM Granite, which is not served through hosted APIs, keeping a distinct open-weight family. For each generator we sample the same 2,500 products from the human pool, so every generator describes the same set. A fixed prompt template supplies the product title, category, rating, and a target word count drawn from the human length distribution; this prevents a detector from exploiting length. We sample with `temperature` = 1.0, `top_p` = 0.95. Per-generator mean word counts (59.9 to 68.6) track the human mean of 60.6.

3.4 Cleaning

A `langdetect` sweep removed 31 non-English human and 11 non-English AI reviews. Reasoning-model leakage required more: 289 Qwen reviews were pure reasoning traces and were deleted (fixed by disabling thinking server-side); 97 DeepSeek reviews leaked `<think>` blocks and were salvaged by stripping them, and 19 truncated ones were dropped. The split builder absorbs the resulting non-round counts.

3.5 Leave-one-generator-out splits

We use four LOGO rotations (Table 2). Each holds out one generator entirely as the cross-generator test set and trains on humans plus the other three.

The three training generators are split 80/10/10 into `train`, `val_indist`, and `test_indist`. The held-out generator plus a disjoint human pool forms `test_crossgen`. We carve humans after the AI split, taking exactly enough humans for a 1:1 in-distribution balance and assigning the rest (about 2,470) to the cross-generator pool. Because these humans are disjoint from training, the cross-generator test measures generalization to an unseen generator, not memorized human text; the builder asserts no `review_id` overlap. Table 3 gives the sizes.

Table 3: Split sizes per rotation. In-distribution splits are exactly 1:1.

Rotation	train	val_indist	test_indist	test_crossgen (AI + human)
0 (gpt5mini out)	11,994	1,496	1,504	4,962 (2,490 + 2,472)
1 (deepseek out)	11,980	1,496	1,500	4,980 (2,499 + 2,481)
2 (gemma out)	11,978	1,494	1,502	4,982 (2,500 + 2,482)
3 (qwen out)	11,982	1,496	1,500	4,978 (2,498 + 2,480)

Table 4: The four DeBERTa variants and their active loss terms.

Variant	Active loss
base (CE)	$\mathcal{L}_{ce}$
contrastive	$\mathcal{L}_{ce} + \lambda_c \mathcal{L}_{supcon}$
adversarial	$\mathcal{L}_{ce} + \lambda_a \mathcal{L}_{adv}$
both	all three

3.6 Model and objectives

The detector is a DeBERTa-v3-base encoder [2] with up to three heads on the mean-pooled representation h . The classification head is a linear layer trained with cross-entropy on the human/AI label; this head is the detector, and the others only shape the encoder. The contrastive head projects h to 128 dimensions and applies supervised contrastive loss [3] on the binary label, clustering AI together and human together across generators. The adversarial head predicts the source (three training generators plus **human**) through a gradient reversal layer [4], so minimizing its loss removes generator-specific cues. The total loss is

$$\mathcal{L} = \mathcal{L}_{ce} + \lambda_c \mathcal{L}_{supcon} + \lambda_a \mathcal{L}_{adv}, \quad (1)$$

with $\lambda_c = \lambda_a = 0.5$ and λ_a ramped from 0 over training for stability. The four variants toggle which terms are active (Table 4).

3.7 Baselines and training

We compare against TF-IDF with logistic regression and a frozen DeBERTa encoder with a logistic-regression probe, each run per rotation. All variants use the configuration in Table 5, select the best epoch on `val_indist` F1, and fix seeds at 42.

4 Experiments and Results

We report F1 (positive class AI) and ROC-AUC on the in-distribution and cross-generator test sets, averaged over four rotations. The headline metric is the gap: in-distribution F1 minus cross-generator F1.

4.1 Baselines

Table 6 shows both baselines. TF-IDF is strong in-distribution (0.9709) but loses 1.5 F1 points cross-generator (gap 0.0151). The frozen-DeBERTa probe is weaker (0.9437) and less stable, with a 0.0589 gap when DeepSeek is held out. Both carry a clear cross-generator penalty.

Table 5: Training configuration, shared across all variants and rotations.

Setting	Value
Learning rate	2e-5
Epochs	3
Batch size	4 (effective 16 with gradient accumulation 4)
Max sequence length	256
Precision	float32 (avoids NaNs on 4060)
Optimizer	AdamW, weight decay 0.01, 100 warmup steps
Seed	42

Table 6: Baseline LOGO results, averaged over four rotations.

Baseline	In-Dist F1	In-Dist AUC	Cross-Gen F1	Gap (F1)
TF-IDF + LogReg	0.9709	0.9964	0.9558	0.0151
Frozen DeBERTa + LogReg	0.9657	0.9950	0.9437	0.0220

4.2 DeBERTa variants

Table 7 is the main result. Fine-tuning lifts cross-generator F1 to 0.9831 or above for every variant, with AUC above 0.995. Contrastive is the best detector: highest cross-generator F1 (0.9875) and AUC (0.9994), and it halves the gap versus cross-entropy (0.0048 versus 0.0095). Adversarial gives the smallest gap (0.0015) but lowers F1, trading accuracy for invariance. Combining both does not beat contrastive alone. Figures 1 and 2 visualize this.

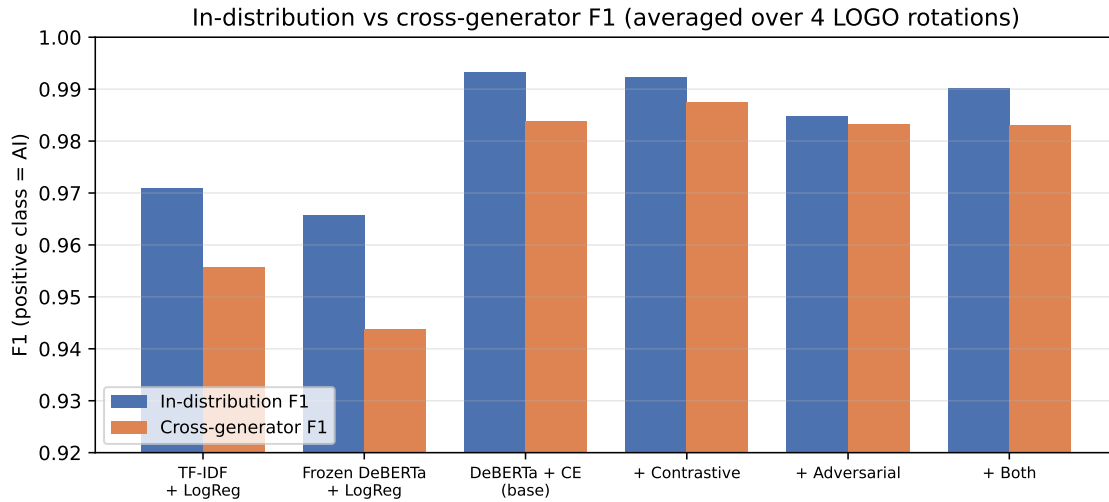


Figure 1: In-distribution versus cross-generator F1 for the baselines and the four DeBERTa variants. Fine-tuned DeBERTa closes most of the baseline gap.

Table 7: Main comparison: DeBERTa variants, averaged over four rotations. Best value per column in bold.

Model	In-Dist F1	Cross-Gen F1	Gap (F1)	Cross-Gen AUC
DeBERTa + CE (base)	0.9933	0.9839	0.0095	0.9991
+ Contrastive	0.9924	0.9875	0.0048	0.9994
+ Adversarial	0.9847	0.9832	0.0015	0.9964
+ Both	0.9901	0.9831	0.0069	0.9954

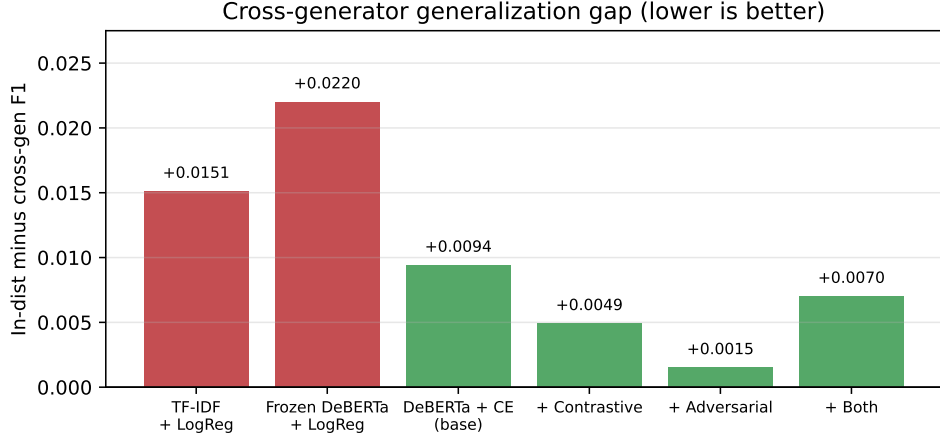


Figure 2: Cross-generator gap (in-distribution F1 minus cross-generator F1). The contrastive and adversarial objectives both shrink it.

4.3 Per-generator breakdown

Table 8 and Figure 3 break cross-generator F1 down by held-out generator. Gemma is the hardest to catch unseen: cross-entropy drops to 0.9681. Both objectives help most here, raising held-out-Gemma F1 to 0.9808 (contrastive) and 0.9815 (adversarial). GPT-5 mini and Qwen are easiest, with all variants above 0.98. This is why contrastive raises the average: its largest gains land where the base model is weakest.

5 Discussion and Analysis

Fine-tuning closes most of the gap. The biggest gain comes from fine-tuning, not the robustness heads: cross-generator F1 rises from 0.9437 (frozen) and 0.9558 (TF-IDF) to 0.9839 for plain cross-entropy. A fine-tuned DeBERTa already learns a fairly generator-invariant notion of AI writing; the extra objectives add a smaller improvement.

Contrastive is the best practical detector. It has the highest cross-generator F1 and AUC and halves the gap without the accuracy cost of the adversarial variant. Clustering all AI together and all human together is exactly the bias the task rewards, and its gains are largest on the hardest rotation (Gemma).

Table 8: Cross-generator F1 by held-out generator. Best value per row in bold.

Rotation	Held out	CE	Contrastive	Adversarial	Both
0	gpt5mini	0.9912	0.9902	0.9875	0.9844
1	deepseek	0.9843	0.9861	0.9831	0.9802
2	gemma	0.9681	0.9808	0.9815	0.9754
3	qwen	0.9918	0.9930	0.9807	0.9926

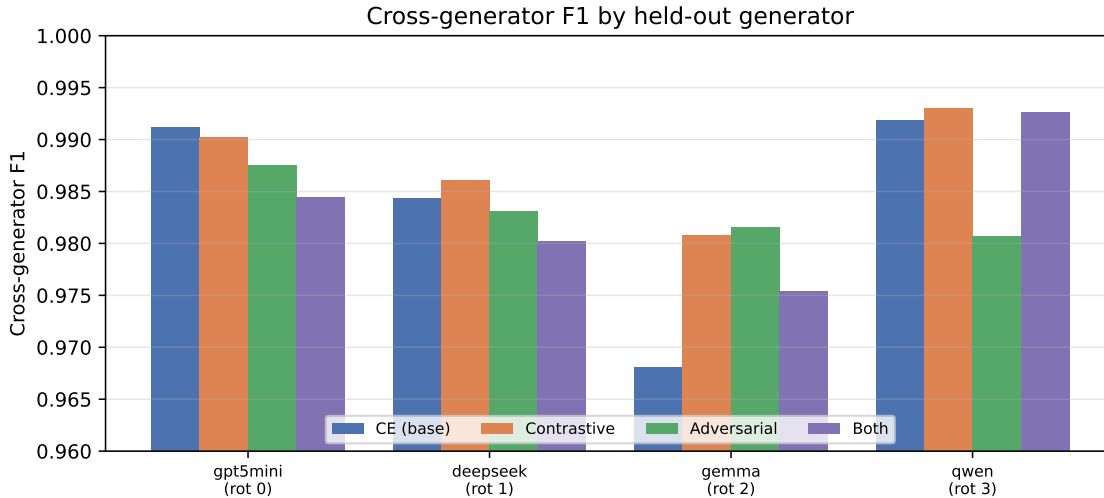


Figure 3: Cross-generator F1 by held-out generator. Gemma is hardest, and is where the contrastive and adversarial objectives help most.

Adversarial minimizes the gap, not the error. The adversarial gap is smallest because it lowers in-distribution F1 toward the cross-generator level, not because it raises cross-generator F1. So a small gap is not the same as a good detector, and the gap should be read together with absolute F1. Combining both objectives did not help, suggesting they interfere at these weights.

Limits of the controlled setting. Because lengths were matched to humans and all generators wrote about the same products, the gap reflects style, not length or topic. That suits the scientific question but makes detection easier than in the wild, where fake reviews often differ in length and cluster on certain products. Our F1 numbers are an upper bound for a controlled setting.

Other limitations. We used DeepSeek instead of the proposal’s Granite, so results speak to the GPT-5 / DeepSeek / Gemma / Qwen family. The corpus is English-only. Three of four generators are reasoning models, and subtle reasoning artifacts could remain as a shortcut. A single fixed prompt was used, so this measures non-adversarial generation. Each cross-generator human pool is modest (about 2,470), adding sampling noise.

6 Conclusion

We asked whether an AI-review detector generalizes to unseen generators, using a controlled corpus of about 20,000 reviews and a strict leave-one-generator-out protocol that fixes product, topic, and

length. Fine-tuned DeBERTa-v3 generalizes far better than TF-IDF or frozen-feature baselines, and a supervised contrastive objective is the best variant: highest cross-generator F1 (0.9875) and AUC (0.9994), and half the gap of plain cross-entropy (0.0048 versus 0.0095). The adversarial objective shrinks the gap further (0.0015) but only by lowering in-distribution accuracy, so the gap must be read alongside absolute error. Gemma is the hardest generator to catch unseen, and is where the robustness objectives help most. Future work should test genuinely held-out model families, prompt-tuned adversarial generation, and non-English reviews.

Contribution Statement

Placeholder: describe each team member’s contributions to data collection, generation, split construction, model implementation, training, evaluation, and writing.

References

- [1] McAuley Lab. Amazon Reviews 2023 dataset. <https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>.
- [2] P. He, X. Liu, J. Gao, and W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *arXiv:2111.09543*, 2021.
- [3] P. Khosla et al. Supervised contrastive learning. In *NeurIPS*, 2020.
- [4] Y. Ganin and V. Lempitsky. Unsupervised domain adaptation by backpropagation. In *ICML*, 2015.
- [5] S. Bag et al. Detecting LLM-generated fake reviews: a leakage-free multi-model framework. *International Journal of Electronic Marketing and Retailing (IJEIM)*, 2025.
- [6] B. Guo et al. DetectRL: benchmarking LLM-generated text detection in real-world scenarios. In *NeurIPS*, 2024.
- [7] M. Kashifpour et al. Trust at risk: detecting misinformation in LLM-generated product reviews. *ScienceDirect*, 2025.
- [8] Wikipedia. Fake review. https://en.wikipedia.org/wiki/Fake_review.
- [9] Wikipedia. Large language model. https://en.wikipedia.org/wiki/Large_language_model.
- [10] Wikipedia. Transformer (deep learning architecture). [https://en.wikipedia.org/wiki/Transformer_\(deep_learning_architecture\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)).
- [11] Wikipedia. Contrastive learning. https://en.wikipedia.org/wiki/Contrastive_learning.